Monte Carlo Pricing of an Up-and-Out Call Option

Subash K C
Uppsala University

January 15, 2026

1 Introduction

The purpose of this project is to price an up-and-out European call option on Google (GOOGL) using Monte Carlo simulation under the risk-neutral measure. Barrier options are path-dependent derivatives whose payoff depends on whether the underlying asset price crosses a predefined barrier during the option's lifetime. Due to this path dependence, closed-form pricing formulas are limited, making Monte Carlo methods particularly suitable.

In addition to pricing, we analyze the sensitivity of the option price to key parameters: the barrier level, strike price, and time to maturity. Volatility is estimated from historical stock prices. Volatility estimation uncertainty is quantified via bootstrap resampling. To improve computational efficiency and reduce estimator variance, two variance reduction techniques are implemented: antithetic variates and control variates. Their performance is compared to assess the efficiency gains in the barrier option setting.

Overall, this project demonstrates how Monte Carlo methods can be used to analyze the sensitivity of an up-and-out European call option to the barrier level, strike price, and maturity. This project also demonstrates how to properly account for estimation uncertainty using bootstrap methods and improve computational efficiency through variance reduction techniques while pricing call options.

2 Stock Dynamics and Up-and-Out Call Payoff

Under the risk-neutral measure, the stock price follows geometric Brownian motion:

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t,$$

where r is the risk-free rate and σ is the volatility. The up-and-out call option with strike K, barrier $B > S_0$, and maturity T has payoff

$$\text{Payoff} = \begin{cases} \max(S_T - K, 0), & \max_{0\le t\le T} S_t < B, \\ 0, & \text{otherwise.} \end{cases}$$

3 Method

3.1 Data And Parameters

Historical daily closing prices for Google (GOOGL) were obtained from Yahoo Finance for the period January 1, 2023 to December 31, 2023 (252 trading days). The option parameters are specified as follows:

Table 1: Option Parameters

Parameter	**Symbol**	**Value**
Spot Price	S_0	\$139.88
Strike Price	K	\$153.87 ($1.10 \times S_0$)
Barrier Level	B	\$174.85 ($1.25 \times S_0$)
Time to Maturity	T	0.5 years
Risk-free Rate	r	5%
Trading Days per Year		252

3.2 Volatility Estimation

Daily log-returns were computed as:

$$R_t = \ln\left(\frac{S_t}{S_{t-1}}\right).$$

and the annualized volatility is estimated by

$$\sigma = \mathrm{std}(R_t)\sqrt{252}.$$

3.2.1 Bootstrap Volatility

To account for uncertainty in the estimated volatility, bootstrap resampling was applied:

1. Daily log-returns were resampled with replacement 1000 times.
2. For each sample, the standard deviation was computed and annualized:

$$\sigma_i = \mathrm{std}(R_t^{(i)})\sqrt{252}.$$

3. The resulting 1000 bootstrapped volatilities provide a distribution capturing estimation uncertainty.

The mean annualized volatility used for GBM simulation was

$$\sigma_{\text{mean}} = 0.3045,$$

with a 95% confidence interval of

$$[0.2590, 0.3529].$$

This approach explicitly accounts for the fact that volatility is estimated from finite historical data.

4 Monte carlo Simulation

4.1 Monte Carlo Simulation Method

The option price is approximated using Monte Carlo simulation as follows:

1. We discretize the time interval $[0, T]$ into N steps of size Δt.
2. Simulate stock price paths using

$$S_{t_{n+1}} = S_{t_n} \exp\left(\left(r - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t} Z_n\right),$$

where $Z_n \sim \mathcal{N}(0, 1)$.

3. Monitor each path for barrier crossing.
4. Compute discounted payoffs for surviving paths.

The option value is then estimated as

$$V_0 \approx e^{-rT} \frac{1}{M} \sum_{i=1}^{M} \text{Payoff}_i.$$

Barrier monitoring is performed at discrete time points, which may underestimate the probability of barrier crossing and thus introduce discretization bias.

4.2 Varaince reduction

To reduce the variance of the Monte Carlo estimator, antithetic variates and Control Variates were used. Variance reduction techniques improve efficiency and reduce the number of simulations required for a given accuracy.

4.2.1 Antithetic Variates

For each standard normal variate $Z \sim \mathcal{N}(0,1)$, we generate two paths using Z and $-Z$. The antithetic estimator is:

$$\hat{V}_{\text{AV}} = \frac{e^{-rT}}{2M} \sum_{i=1}^{M} \left[V(S^{(+,i)}) + V(S^{(-,i)}) \right]$$

where $S^{(+,i)}$ and $S^{(-,i)}$ are paths generated with Z and $-Z$ respectively.

4.2.2 Control Variates

We use the vanilla European call option as a control variate. Let Y_i be the discounted barrier option payoff and X_i be the discounted vanilla call payoff from the same simulated path i. The Black-Scholes price C_{BS} provides the known expectation $\mathbb{E}[X]$. The control variate estimator is:

$$\hat{V}_{\text{CV}} = \frac{1}{M} \sum_{i=1}^{M} \left[Y_i + \beta^* (X_i - C_{\text{BS}}) \right],$$

where

$$\beta^* = -\frac{\text{Cov}(Y,X)}{\text{Var}(X)}, \quad \hat{\beta} = -\frac{\sum_{i=1}^{M} (Y_i - \bar{Y})(X_i - \bar{X})}{\sum_{i=1}^{M} (X_i - \bar{X})^2}.$$

The variance reduction factor is $1 - \rho_{Y,X}^2$, where $\rho_{Y,X}$ is the correlation between Y and X.

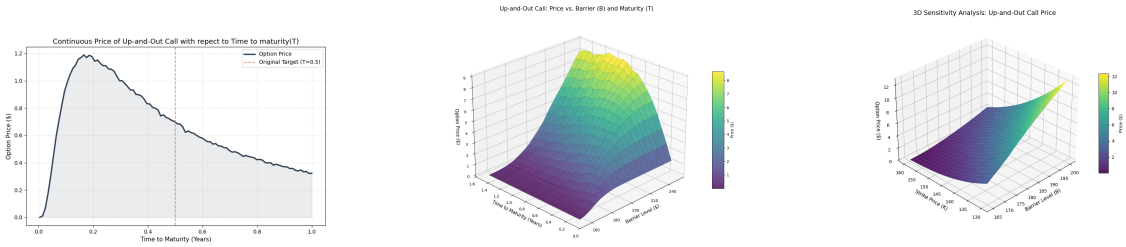
5 Results and Discussion

5.1 Volatility estimation

The pricing analysis of the Google (GOOGL) up-and-out call option shows a strong dependence on the contract parameters market volatility. As seen in Figure 1, the option price drops sharply as the spot approaches the 174.85 barrier, reflecting the increasing probability of a knock-out event.

Unlike a standard call option, whose value typically increases with maturity, the up-and-out call exhibits a non-monotonic relationship with time. For short maturities, added time increases the premium, but for longer maturities the growing likelihood of hitting the barrier reduces the

option value toward zero. The high annualized volatility of $\sigma = 0.3045$ further amplifies this effect. While higher volatility generally increases the potential upside of the underlying assets, it also raises the probability that the barrier will be breached, which in this case significantly suppresses the option price. This "double-edged sword" effect makes the contract particularly sensitive to both barrier proximity and Maturity(T), resulting in the relatively low fair value observed in the simulations.
Both the barrier levels and strike price also significantly affect the price. A barrier set far above the spot makes the option behave more like a standard call, while a higher strike lowers the premium. Together, these parameters shape the overall pricing behavior of the option.



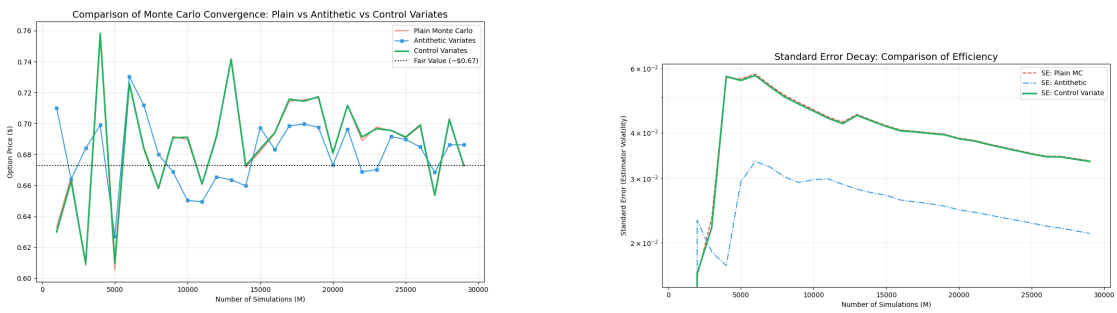
(a) Option price as a function of Maturity(T)

(b) Option price as a function of T and Barrier(B)

(c) Option price as a function of Barrier and strike price.

Figure 1: Sensitivity Analysis. Plot (a) shows the non-monotonic relationship with maturity; (b) and (c) Illustrate the option price as a funcition of Maturity(T) and Barrier(B) and strike price(K)

The performance of the variance reduction techniques is shown in Figure 6. The Antithetic Variate method consistently achieves a lower standard error than both Plain Monte Carlo and Control Variates, reducing it by roughly a factor of 2–3, which highlights its robustness in the near-barrier regime. In contrast, the Control Variate method performs similarly to Plain Monte Carlo when the barrier is close to the spot due to the weak correlation with the vanilla call payoff. As the barrier moves further away (e.g., B = 300), the knock-out probability drops, the barrier option behaves more like a standard call, and the Control Variate regains its efficiency advantage.
The convergence plot (Figure 6a) and Standard error decay plot (Figure 6b) illustrates these differences: Antithetic Variates stabilize quickly around the true option price, while Plain Monte Carlo and control Variates fluctuates more. In the standard error decay plot, it is evident that the error for Antithetic Variates remains consistently lower than for the other two methods, demonstrating its superior efficiency, especially near the barrier.

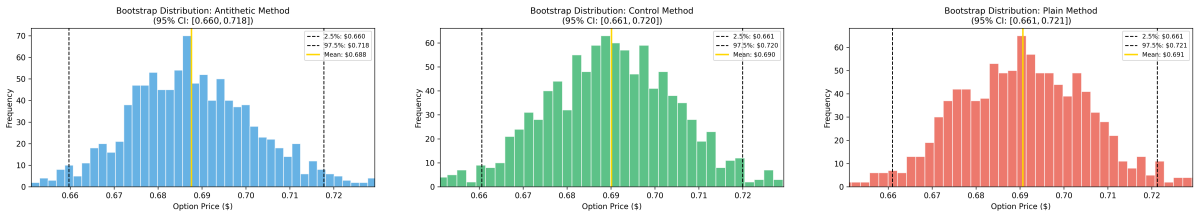


(a) Convergence plot

(b) Standard error decay

Figure 2: Convergence Analysis Plot (a) shows the convergence of the differenct approach (b) illustrate standard error for the different approach.

Bootstrap resampling (10,000 iterations at $M = 30,000$ simulations, Figure 3) further quantifies this uncertainty. The Antithetic distribution is sharply concentrated, yielding a 95% confidence interval of $[0.660, 0.718]$, whereas Control Variate and Plain Monte Carlo distributions remain wider. The narrow interval for Antithetic Variates confirms its robustness, particularly important given the combined effect of high volatility and knock-out risk on the option's value.



(a) Bootstrap distribution of Antithetic variaties

(b) Bootstrap distribution of control variaties

(c) Bootstrap distribution of plain monte carlo.

Figure 3: Bootsrap Distribution: (a) (b) and (c) Illustrate the bootsrap distribution of the monte carlo simulation

6 Conclusion

This study demonstrates the application of Monte Carlo simulation for pricing path-dependent options, specifically up-and-out calls on GOOGL. The results highlight the strong sensitivity of the option price to contract parameters, including the barrier, strike, and time to maturity. High underlying volatility significantly suppresses the option value due to the increased probability of hitting the barrier, illustrating the effect of volatility in barrier option pricing.
Variance reduction techniques proved essential for obtaining reliable estimates. Antithetic Variates consistently delivered the lowest standard error and the narrowest bootstrap confidence intervals, particularly near the barrier, confirming their robustness in challenging simulation regimes. Control Variates were effective when the barrier was distant but lost efficiency near the spot, reflecting the dependence on correlation with the vanilla call payoff.
Overall, the project demonstrates that careful modeling of volatility, barrier effects, and variance reduction methods is critical for accurately pricing barrier options. However, A significant challenge in pricing continuous barrier options with Monte Carlo is the monitoring bias. In a standard simulation, we only check if the stock price hits the barrier at specific time steps. However, it is entirely possible for the stock price to "jump" over the barrier and back between two discrete observations. Standard Monte Carlo would treat this path as active, while in reality, it should have been "knocked out". This issue can be mitigated using Brownian Bridge interpolation, which estimates the probability of barrier crossings between time steps and provides a more accurate assessment of the knock-out risk, improving the precision of the Monte Carlo estimate.

A Appendix

The following plots provide additional empirical evidence supporting the study's findings but were omitted from the main text and should not be included in the page count.

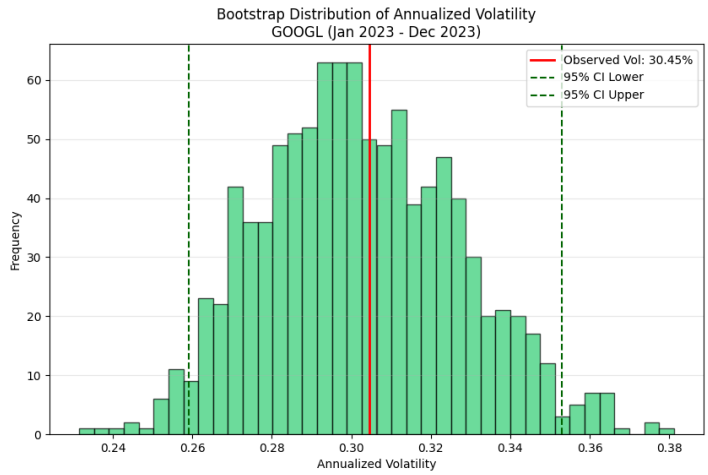


Figure 4: Bootstrap distribution of Volatility Google.

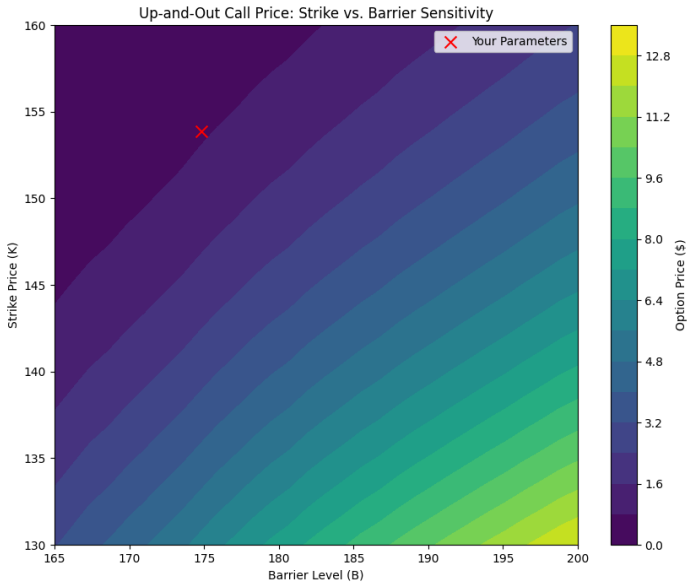
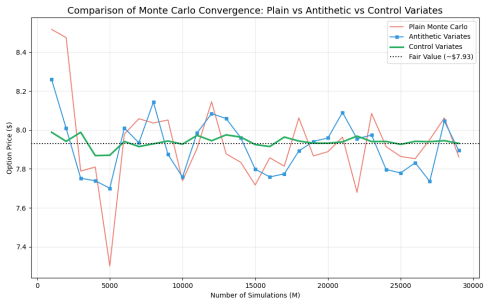
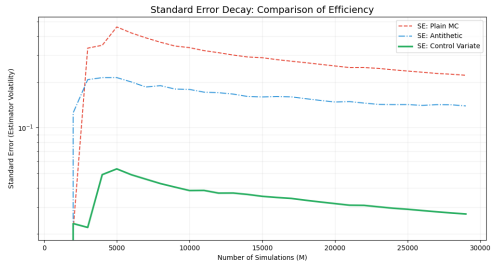


Figure 5: Heat map of strike price and Barrier levels for option price.



(a) Convergence plot



(b) Standard error decay

Figure 6: Convergence Analysis Plot for Higher Barrier (a) shows the convergence of the differenct approach (b) illustrate standard error for the different approach.

B Python Implementation

The full Python script used for data acquisition, volatility estimation, Monte Carlo simulation, and bootstrap analysis is provided in the appendix and should not be included in the page count.

```python
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt



# data from yahoo finance
ticker = "GOOGL"
data = yf.download(ticker, start="2023-01-01", end="2023-12-31",
    auto_adjust=False)



# Calculate daily log returns using the "Close"

data['Returns'] = np.log(data['Close'] / data['Close'].shift(1))
returns = data['Returns'].dropna()

# Annualized Volatility function
def calc_annualized_vol(rev_series):
    # Standard deviation of returns * square root of trading days (252)
    return rev_series.std() * np.sqrt(252)

obs_vol = calc_annualized_vol(returns)
print(f"Observed Annualized Volatility (Close Price, 2023): {obs_vol
    :.2%}")

#Bootstrap the volatility (1000 iterations)
```

```python
n_iterations = 1000
bootstrap_vols = []

# Using a fixed seed for reproducibility
np.random.seed(42)

for _ in range(n_iterations):
    # Resample with replacement to simulate uncertainty
    resample = np.random.choice(returns, size=len(returns), replace=True)
    vol = calc_annualized_vol(pd.Series(resample))
    bootstrap_vols.append(vol)

# 5. Calculate 95% Confidence Intervals (2.5th and 97.5th percentiles)
ci_lower = np.percentile(bootstrap_vols, 2.5)
ci_upper = np.percentile(bootstrap_vols, 97.5)

print(f"95% Confidence Interval for Volatility: [{ci_lower:.2%}, {ci_upper:.2%}]")

# plot
plt.figure(figsize=(10, 6))
plt.hist(bootstrap_vols, bins=40, color='#2ecc71', edgecolor='black', alpha=0.7)
plt.axvline(obs_vol, color='red', lw=2, label=f'Observed Vol: {obs_vol:.2%}')
plt.axvline(ci_lower, color='darkgreen', ls='--', label='95% CI Lower')
plt.axvline(ci_upper, color='darkgreen', ls='--', label='95% CI Upper')

plt.title(f'Bootstrap Distribution of Annualized Volatility\n{ticker} (Jan 2023 - Dec 2023)')
plt.xlabel('Annualized Volatility')
plt.ylabel('Frequency')
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()




# plain monte carlo oricing function

def price_up_and_out_call(S0, K, B, T, r, sigma, n_sims=100000):
    """
    Prices an Up-and-Out European Call using Monte Carlo Simulation.
    """
    if S0 >= B: return 0.0  # Already knocked out

    n_days_per_year = 252
    n_steps = int(T * n_days_per_year)
    if n_steps <= 0: return max(S0 - K, 0) # Instant payoff at maturity

    dt = T / n_steps

    # Simulate Paths
    np.random.seed(42)
    Z = np.random.standard_normal((n_sims, n_steps))
    drift = (r - 0.5 * sigma**2) * dt
```

```python
    diffusion = sigma * np.sqrt(dt)

    log_returns = drift + diffusion * Z
    path_returns = np.column_stack([np.zeros(n_sims), log_returns])
    paths = S0 * np.exp(np.cumsum(path_returns, axis=1))

    # Barrier Logic
    max_prices = np.max(paths, axis=1)
    knocked_out = max_prices >= B

    # Payoff Calculation
    ST = paths[:, -1]
    payoffs = np.maximum(ST - K, 0)
    payoffs[knocked_out] = 0

    # Discount to Present Value
    price = np.exp(-r * T) * np.mean(payoffs)
    return price


#Parameters
params = {
    "S0": 139.88,
    "K": 153.87,
    "B": 174.85,
    "r": 0.05,
    "sigma": obs_vol,
}

# Extract parameters for the list comprehension
S0 = params["S0"]
K = params["K"]
B = params["B"]
r = params["r"]


# calculating the final value
final_val = price_up_and_out_call(T=0.5, **params)
print(f"Price for T=0.5: ${final_val:.4f}")




# Monte carlo using Antithetic Variates

def price_up_and_out_antithetic(S0, K, B, T, r, sigma, n_sims=50000):
    """
    Prices an Up-and-Out Call using Antithetic Variates for variance reduction.
    """
    if S0 >= B: return 0.0

    n_steps = int(T * 252)
    dt = T / n_steps

    # Generate random shocks (Z) for HALF the number of simulations
    # because each Z will create two paths (Z and -Z)
```

```python
    half_sims = n_sims // 2
    np.random.seed(42)
    Z = np.random.standard_normal((half_sims, n_steps))


    Z_antithetic = -Z

    # Combining into one matrix to process everything at once (Vectorized)
    Z_full = np.vstack([Z, Z_antithetic])

    # Path Generation
    drift = (r - 0.5 * sigma**2) * dt
    diffusion = sigma * np.sqrt(dt)

    log_returns = drift + diffusion * Z_full
    paths = S0 * np.exp(np.cumsum(log_returns, axis=1))

    # Barrier & Payoff
    # Checking for barrier breach across all 2M paths
    hit_barrier = np.any(paths >= B, axis=1)

    ST = paths[:, -1]
    payoffs = np.maximum(ST - K, 0)
    payoffs[hit_barrier] = 0

    # Average and Discount
    # The average is taken over all n_sims (which are paired)
    price = np.exp(-r * T) * np.mean(payoffs)

    # Return both the price and the standard error
    std_error = np.std(payoffs) / np.sqrt(n_sims)

    return price, std_error


# Monte carlo using control variaties

from scipy.stats import norm

def black_scholes_vanilla(S0, K, T, r, sigma):
    """Calculates the theoretical Black-Scholes price for a Vanilla Call."""
    d1 = (np.log(S0 / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S0 * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

def price_control_variate(S0, K, B, T, r, sigma, n_sims=50000):
    n_steps = int(T * 252)
    dt = T / n_steps

    # Simulating Paths
    np.random.seed(42)
    Z = np.random.standard_normal((n_sims, n_steps))
    drift = (r - 0.5 * sigma**2) * dt
    diffusion = sigma * np.sqrt(dt)
    paths = S0 * np.exp(np.cumsum(drift + diffusion * Z, axis=1))
```

```python

    # Payoffs
    ST = paths[:, -1]

    # Y = Barrier Payoffs (Target)
    hit_barrier = np.any(paths >= B, axis=1)
    Y = np.exp(-r * T) * np.maximum(ST - K, 0)
    Y[hit_barrier] = 0

    # X = Vanilla Payoffs (Control)
    X = np.exp(-r * T) * np.maximum(ST - K, 0)

    # Calculateing Optimal Control Coefficient (cstar)
    # cstar = - Cov(Y, X) / Var(X)
    cov_matrix = np.cov(Y, X)
    c_star = - cov_matrix[0, 1] / cov_matrix[1, 1]

    #Black-Scholes Vanilla Price (Expectation of X)
    C_BS = black_scholes_vanilla(S0, K, T, r, sigma)

    # Control Variate Estimate
    # CV_Payoffs = Y + cstar(X - C_BS)
    cv_payoffs = Y + c_star * (X - C_BS)

    return np.mean(cv_payoffs), np.std(cv_payoffs) / np.sqrt(n_sims)




# Bootrap option price


n_bootstrap = 500
mc_prices = []

for i in range(n_bootstrap):
    np.random.seed(500)

    #change depeding on which you want to  bootsrap plain, Antithetic or control
    price = price_up_and_out_call(T=0.5, **params)
    mc_prices.append(price)

mc_lower = np.percentile(mc_prices, 2.5)
mc_upper = np.percentile(mc_prices, 97.5)

print("Plain MC Bootstrap CI:")
print(f"95% CI: [{mc_lower:.4f}, {mc_upper:.4f}]")
```

Listing 1: GOOGL Option Pricing and Variance Reduction